

SAE 102 : communautés dans un réseau

Cette SAE est la suite de la S101, à faire en binôme et à rendre pour le 21 janvier. Pour le contrôle il faut connaître les structures et les fonctions utilisées en S101 et S102.

Les fonctions seront implémentées dans un module appelé `comu.py` et les tests unitaires dans un module `test_comu.py`. Les comparaisons théoriques et pratiques sont à faire dans un Jupyter notebook `compare_comu.ipynb`. Enfin, le répertoire du projet doit contenir un fichier `binome.txt` avec les noms des étudiants du binôme, un par ligne.

Définitions

Un groupe est un tableau de chaînes de caractères qui sont les noms des personnes du groupe. Par exemple `["Alice", "Bob"]` est un groupe de deux personnes.

Un réseau est un dictionnaire dont les clés sont les noms des personnes et les valeurs les tableaux de leurs amis. Par exemple

```
{
  "Alice" : ["Bob", "Dan"],
  "Bob"   : ["Alice", "Carl", "Dan"],
  "Carl"  : ["Bob"],
  "Dan"   : ["Alice", "Bob"]
}
```

est un réseau de 4 personnes : Alice, Bob, Carl et Dan. Bob est l'ami de tous, Alice est aussi amie de Dan. On pourra dessiner le réseau. Remarque, l'ordre des amis n'a pas d'importance : si les amis d'Alice étaient `["Dan", "Bob"]` le réseau serait le même.

Une communauté est un groupe de personnes qui sont toutes amies entre elles : quand on prend deux membres du groupe, ils sont amis. Dans l'exemple `["Alice", "Bob", "Dan"]` est une communauté ; `["Alice", "Bob", "Carl"]` n'en est pas une.

Questions

- 1) Définir une fonction `cree_reseau` semblable à la fonction `dico_reseau` de la SAE 101 : elle prend en paramètre un tableau de couples d'amis qu'elle traverse une seule fois, et pour chaque couple `(a, b)` ajoute `b` à la liste des amis de `a`, et `a` à la liste des amis de `b`. Elle renvoie le réseau correspondant.
- 2) Comparer théoriquement et pratiquement les fonctions `cree_reseau` et `dico_reseau`. (Cette dernière construit le réseau en cherchant d'abord les amis d'Alice, puis de Bob, etc.)
- 3) Définir la fonction `liste_personnes` qui à partir d'un réseau renvoie la liste des membres du réseau dans un tableau. Avec le réseau donné en exemple, elle doit renvoyer `["Alice", "Bob", "Carl", "Dan"]` (l'ordre n'a pas d'importance).
- 4) Définir la fonction `sont_amis` qui teste si deux membres du réseau sont amis. Pour "Alice" et "Bob" de l'exemple, elle renvoie `True`. Pour "Alice" et "Carl" elle renvoie `False`.
- 5) Définir la fonction `sont_amis_de` qui prend une personne et un groupe (et le réseau), et teste si tous les membres du groupe sont amis de cette personne. Pour "Alice" et le groupe `["Bob", "Dan"]` elle renvoie `True`, et `False` pour "Alice" et le groupe `["Bob", "Carl"]`.
- 6) Définir la fonction `est_comu` qui teste si un groupe du réseau est une communauté. (Groupe et réseau doivent être en paramètre.) Par exemple, c'est le cas de `["Alice", "Bob", "Dan"]` mais pas de `["Alice", "Bob", "Carl"]`.

→

7) Dans un groupe on peut trouver une communauté maximale, au sens où personne ne peut y être ajouté. Définir la fonction `comu` qui part d'une communauté vide et examine successivement les membres du groupe passé en paramètre. Chacun est ajouté à la communauté s'il est ami de tous les membres de la communauté déjà créée.

Avec le réseau en exemple, elle renvoie :

- `["Alice", "Bob", "Dan"]` pour le groupe `["Alice", "Bob", "Carl", "Dan"]`
- `["Carl", "Bob"]` pour le groupe `["Carl", "Alice", "Bob", "Dan"]`
- `["Carl"]` pour le groupe `["Carl", "Alice", "Dan"]`

8) Définir la fonction `tri_popu` qui trie un groupe par popularité (nombre d'amis) décroissante. (Le réseau doit évidemment être en paramètre.) Ainsi le tri du groupe `["Alice", "Bob", "Carl"]` donne `["Bob", "Alice", "Carl"]`.

9) Définir la fonction `comu_dans_reseau` qui trie les membres d'un réseau par popularité décroissante, applique l'algorithme de construction de la question 7 et renvoie la communauté obtenue. Avec le réseau en exemple, elle renvoie `["Bob", "Alice", "Dan"]` (l'ordre n'a pas d'importance).

10) On construit maintenant une communauté maximale d'une autre façon.

Définir la fonction `comu_dans_amis` qui part d'une communauté réduite à une personne et examine ses amis dans l'ordre de popularité décroissante. Comme avant, chacun est ajouté à la communauté s'il est ami de tous les membres de la communauté déjà créée.

Si on part d'une personne des plus populaires, l'algorithme est semblable au précédent (les communautés créées peuvent différer si des personnes ont le même nombre d'amis). En partant d' "Alice" on renvoie `["Alice", "Bob", "Dan"]`; en partant de "Carl" on renvoie `["Carl", "Bob"]`.

11) Comparer théoriquement et pratiquement les fonctions `comu_dans_reseau` et `comu_dans_amis` appliquée à une personne des plus populaires (la recherche de la personne la plus populaire sera prise en compte dans la complexité).

12) Définir la fonction `comu_max` qui applique `comu_dans_amis` à tous les membres d'un réseau et renvoie la plus grande communauté trouvée. Avec le réseau en exemple elle renvoie `["Alice", "Bob", "Dan"]` (peu importe l'ordre).